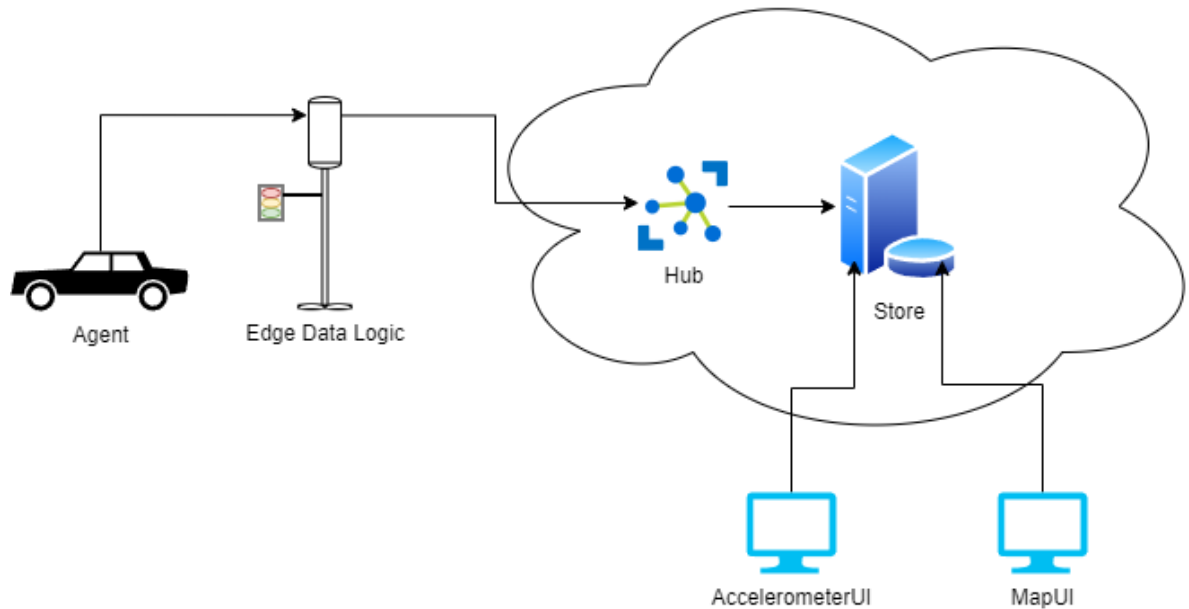


Лабораторна робота №1

Опис проекту



Завдання – розробити комплексну систему моніторингу та аналізу стану дорожнього покриття. Система може мати можливість збирати дані про різні параметри, такі як нерівності, тріщини, вибоїни та інші дефекти, які можуть вплинути на якість дороги. Зібрані дані слід проаналізувати, щоб визначити закономірності та тенденції, які можуть допомогти у прогнозуванні потреб у технічному обслуговуванні дорожньої мережі. Система також може мати можливість генерувати звіти та сповіщення для відповідних органів влади та/або водіїв.

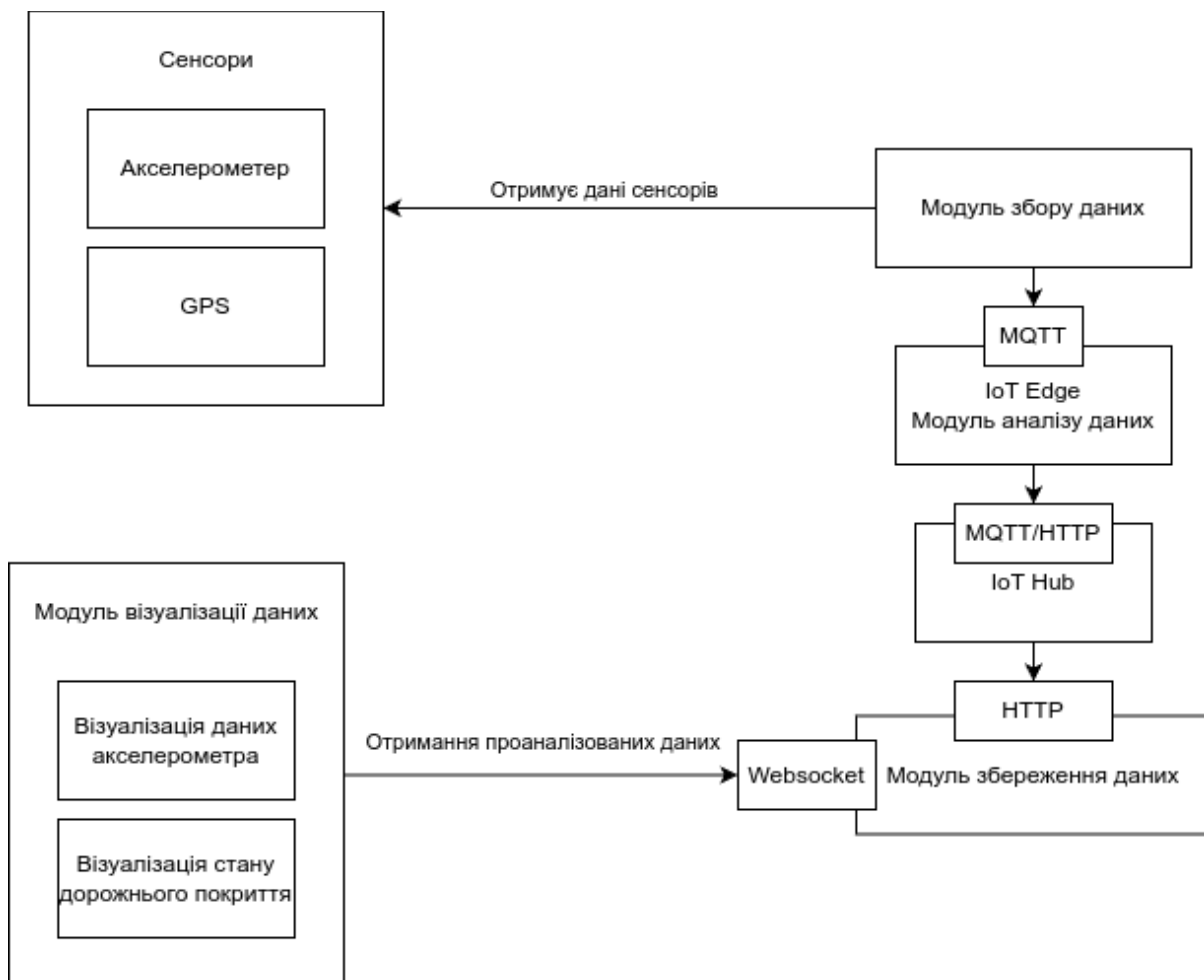
Метою цієї системи є підвищення безпеки дорожнього руху, зниження витрат на технічне обслуговування та покращення загального досвіду водіння для автомобілістів.

В систему входять пристрій на машині (stm32), edge пристрій, хмарні або on-prem сервіси.

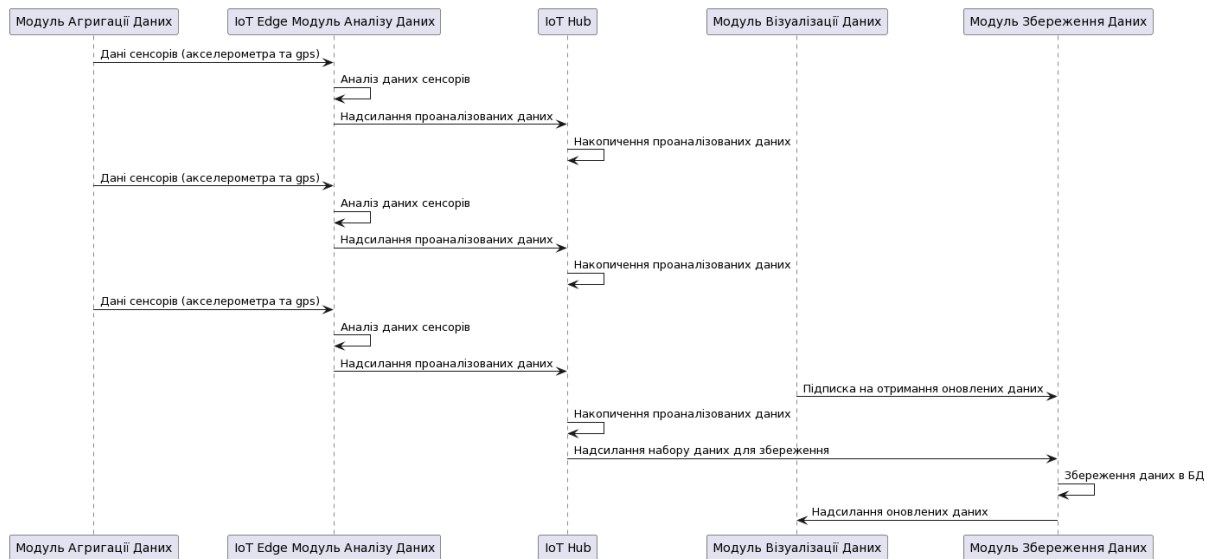
Система містить наступні частини (дозволяється розширювати):

- Модуль збору даних (Agent) - програма агент на машині, яка знімає показники сенсорів (наприклад раз в 100мс) та відправляє їх на сервер.

- Модуль первинної обробки даних (Edge Data Logic) - програма яка займається первинною обробкою даних, в нашому випадку визначає стан дорожнього покриття та відправляє проаналізовані дані на Hub.
- Модуль накопичення даних (Hub) - сервіс який займається накопиченням та аналізом (preprocessing) отриманих даних від Agent та подальшим зберіганням оброблених даних в БД. Перед збереженням дани потрібно накопичити (наприклад 10 записів), так як такі дані простіше аналізувати і запис в БД множини даних буде швидшим.
- Store - арі для отримання та збереження даних в БД.
- Модуль візуалізації (Accelerometer UI, Map UI) - інтерфейс візуалізації (моніторингу) результатів системи.



Архітектура системи



Юзкейс роботи

Постановка задачі

Відбувається розробка програмного забезпечення, яке є частиною великої системи Розумного Міста. А саме розробка Agent частини, яка збирає дані отримані з датчиків вбудованих в сучасні машини для моніторингу стану дорожнього покриття. Проте не всі розробники маю доступ до датчиків для роботи з ними.

Я як розробник хочу мати змогу розробляти систему без фізичного пристрою, датчиків (акселерометер) які видають інформацію.

Потрібно абстрагуватися від конкретного джерела інформації або емулювати роботу датчиків.

Для рішення цієї задачі потрібно реалізувати програмну частину яка буде читати дані датчиків з файлу (до цього записаний csv file під назвою data.csv) та надсилати їх на Edge. В нашому випадку Agent та Edge будуть комунікувати через MQTT.

Кроки виконання

Надалі буде показана реалізація на python, проте ніхто не забороняє використовувати той технологічний стек який для вас є оптимальним.

До початку роботи потрібно встановити [Docker](#) та [MQTT Explorer](#).

1. Створення віртуального середовища. Пакет **virtualenv** повинен бути встановлений в системі.

```
python -m venv venv
```

2. Активація середовища

```
source ./venv/bin/activate # linux  
.\venv\Scripts\activate.bat # windows
```

3. Встановимо необхідні залежності(бібліотеки)

```
pip install paho-mqtt==1.6.1 # mqtt client  
pip install marshmallow # for schema and converting object to json
```

4. Збережемо залежності в requirements.txt

```
pip freeze > requirements.txt
```

5. Структура проекту буде наступною

```
src  
    main.py  
    config.py  
    file_datasource.py  
    schema  
        accelerometer_schema.py  
        gps_schema.py  
        aggregated_data_schema.py  
    domain  
        accelerometer.py  
        gps.py
```

aggregated_data.py
docker
mosquitto/config/mosquitto.conf
docker-compose.yaml
Dockerfile
requirements.txt
README.md

6. Реалізуємо моделі даних.

accelerometer.py

```
1  from dataclasses import dataclass
2
3
4  @dataclass
5  class Accelerometer:
6      x: int
7      y: int
8      z: int
```

gps.py

```
1  from dataclasses import dataclass
2
3
4  @dataclass
5  class Gps:
6      longitude: float
7      latitude: float
```

aggregated_data.py

```

1  from dataclasses import dataclass
2
3  from datetime import datetime
4  from domain.accelerometer import Accelerometer
5  from domain.gps import Gps
6
7
8  @dataclass
9  class AggregatedData:
10     accelerometer: Accelerometer
11     gps: Gps
12     time: datetime

```

7. Наступним кроком реалізуємо схеми які будемо використовувати для конвертації об'єктів в json формат.

accelerometer_schema.py

```

1  from marshmallow import Schema, fields
2  from domain.accelerometer import Accelerometer
3
4
5  class AccelerometerSchema(Schema):
6     x = fields.Int()
7     y = fields.Int()
8     z = fields.Int()

```

location_schema.py

```

1  from marshmallow import Schema, fields
2  from domain.gps import Gps
3
4
5  class GpsSchema(Schema):
6     longitude = fields.Number()
7     latitude = fields.Number()

```

aggregated_data_schema.py

```

1  from marshmallow import Schema, fields
2  from schema.accelerometer_schema import AccelerometerSchema
3  from schema.gps_schema import GpsSchema
4  from domain.aggregated_data import AggregatedData
5
6
7  class AggregatedDataSchema(Schema):
8      accelerometer = fields.Nested(AccelerometerSchema)
9      gps = fields.Nested(GpsSchema)
10     time = fields.DateTime('iso')

```

8. Реалізуємо файл конфігурацій

config.py

```

1  import os
2
3
4  def try_parse(type, value: str):
5      try:
6          return type(value)
7      except Exception:
8          return None
9
10
11     # MQTT config
12     MQTT_BROKER_HOST = os.environ.get('MQTT_BROKER_HOST') or 'mqtt'
13     MQTT_BROKER_PORT = try_parse(int, os.environ.get('MQTT_BROKER_PORT')) or 1883
14     MQTT_TOPIC = os.environ.get('MQTT_TOPIC') or 'agent'
15
16     # Delay for sending data to mqtt in seconds
17     DELAY = try_parse(float, os.environ.get('DELAY')) or 1

```

9. Основну логіку, а саме file_datasource.py потрібно реалізувати самостійно. Перед читанням даних відкрити файли, при читанні зчитувати відповідний набір даних з файлів та повертати його.

file_datasource.py

```

1  from csv import reader
2  from datetime import datetime
3  from domain.aggregated_data import AggregatedData
4
5
6  class FileDatasource:
7      def __init__(self, accelerometer_filename: str, gps_filename: str) -> None:
8          pass
9
10     def read(self) -> AggregatedData:
11         """Метод повертає дані отримані з датчиків"""
12
13     def startReading(self, *args, **kwargs):
14         """Метод повинен викликатись перед початком читання даних"""
15
16     def stopReading(self, *args, **kwargs):
17         """Метод повинен викликатись для закінчення читання даних"""

```

10. Реалізуємо основний файл для запуску програми
main.py


```

1  from paho.mqtt import client as mqtt_client
2  import json
3  import time
4  from schema.aggregated_data_schema import AggregatedDataSchema
5  from file_datasource import FileDatasource
6  import config
7
8  def connect_mqtt(broker, port):
9      """Create MQTT client"""
10     print(f"CONNECT TO {broker}:{port}")
11     def on_connect(client, userdata, flags, rc):
12         if rc == 0:
13             print(f"Connected to MQTT Broker ({broker}:{port})!")
14         else:
15             print(f"Failed to connect {broker}:{port}, return code {rc}\n", rc)
16             exit(rc) # Stop execution
17     client = mqtt_client.Client()
18     client.on_connect = on_connect
19     client.connect(broker, port)
20     client.loop_start()
21     return client
22
23 def publish(client, topic, datasource, delay):
24     datasource.startReading()
25     while True:
26         time.sleep(delay)
27         data = datasource.read()
28         msg = AggregatedDataSchema().dumps(data)
29         result = client.publish(topic, msg)
30         # result: [0, 1]
31         status = result[0]
32         if status == 0:
33             pass
34             # print(f"Send `{msg}` to topic `{topic}`")
35         else:
36             print(f"Failed to send message to topic {topic}")
37
38 def run():
39     # Prepare mqtt client
40     client = connect_mqtt(config.MQTT_BROKER_HOST, config.MQTT_BROKER_PORT)
41     # Prepare datasource
42     datasource = FileDatasource("data/accelerometer.csv", "data/gps.csv")
43     # Infinity publish data
44     publish(client, config.MQTT_TOPIC, datasource, config.DELAY)
45
46 if __name__ == '__main__':
47     run()

```

11.Добавимо Dockerfile для запуску даного агента в docker.

Dockerfile

```
1  # set base image (host OS)
2  FROM python:latest
3  # set the working directory in the container
4  WORKDIR /usr/agent
5  # copy the dependencies file to the working directory
6  COPY requirements.txt .
7  # install dependencies
8  RUN pip install -r requirements.txt
9  # copy the content of the local src directory to the working directory
10 COPY src/ .
11 # command to run on container start
12 CMD ["python", "main.py"]
```

12.Добавимо mosquitto.conf для запуску mqtt.

mosquitto.conf

```
1  persistence true
2  persistence_location /mosquitto/data/
3  listener 1883
4  ## Authentication ##
5  allow_anonymous true
6  # allow_anonymous false
7  # password_file /mosquitto/config/password.txt
8  ## Log ##
9  log_dest file /mosquitto/log/mosquitto.log
10 log_dest stdout
11 # listener 1883
```

13.Налаштуємо docker-compose.yaml для локального запуску та тестування програми.

docker-compose.yaml

```
1  version: "3.9"
2  name: "road_vision"
3  services:
4    mqtt:
5      image: eclipse-mosquitto
6      container_name: mqtt
7      volumes:
8        - ./mosquitto:/mosquitto
9        - ./mosquitto/data:/mosquitto/data
10       - ./mosquitto/log:/mosquitto/log
11     ports:
12       - 1883:1883
13       - 9001:9001
14     networks:
15       mqtt_network:
16
17     fake_agent:
18       container_name: agent
19       build: ../
20       depends_on:
21         - mqtt
22       environment:
23         MQTT_BROKER_HOST: "mqtt"
24         MQTT_BROKER_PORT: 1883
25         MQTT_TOPIC: "agent_data_topic"
26         DELAY: 0.1
27       networks:
28         mqtt_network:
29
30
31     networks:
32       mqtt_network:
```

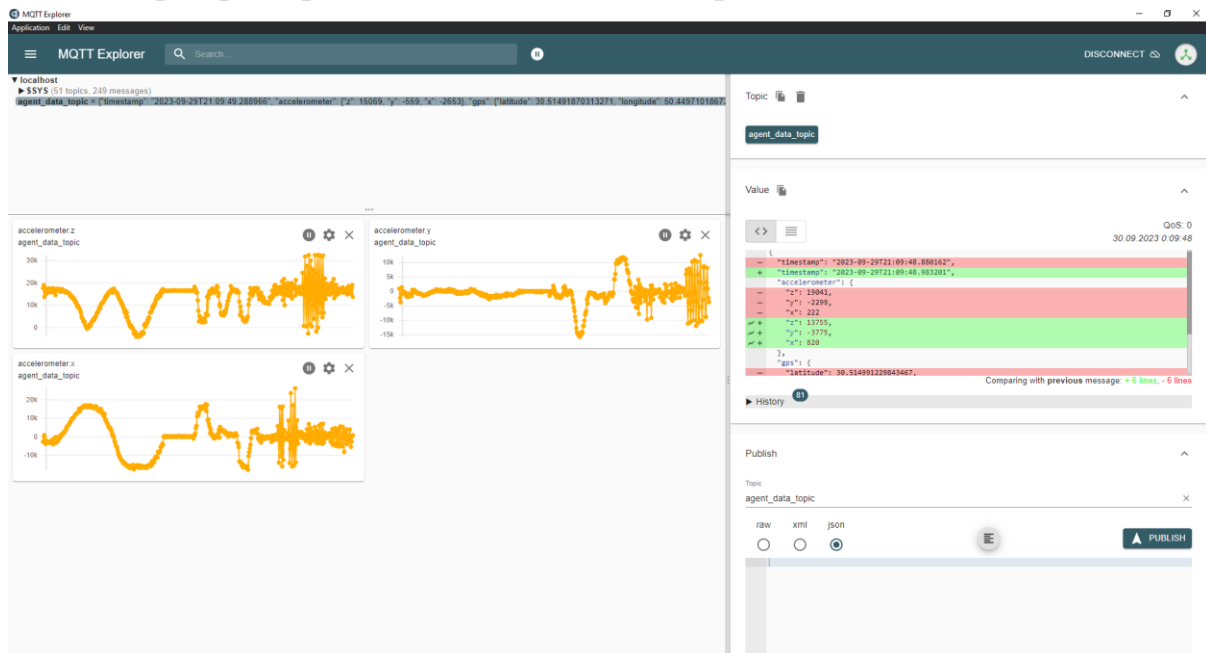
14. Тестові дані(data.csv, gps_data.csv) скачати за посиланням
https://github.com/Toolf/project_template

15. Запуск програму та mqtt в docker

cd docker

docker-compose up --build

16. Перевірка працездатності в MQTT Explorer



Додавання нового датчику в проєкт

1. Для додавання нових датчиків потрібно описати схему даних датчиків на агенті та об'єкт прикладної області, розглянемо на прикладі паркінгу

parking_schema.py

```
1 from agent.src.schema.gps_schema import GpsSchema
2 from marshmallow import Schema, fields
3
4
5 class ParkingSchema(Schema):
6     empty_count = fields.Number()
7     gps = fields.Nested(GpsSchema)
```

parking.py

```
1  from agent.src.domain.gps import Gps
2
3
4  class Parking:
5      empty_count: int
6      gps: Gps
```

2. Додати таблицю даних parking.csv
3. Розширити клас FileDatasource для читання даних з файлу parking_filename та відповідно розширити методи read, startReading, stopReading
4. Розширити логіку в main.py для відправки даних в відповідні топіки

Очікувані результати в протоколі

1. Посилання на git репозиторій з проектом
2. Реалізація FileDatasource
3. Показ результатів роботи в MQTT Explorer
4. Висновки

Ідеї для підвищення оцінки

Реалізація поступового читання (ф-ія read має при кожному зверненні певну повертати набір даних, наприклад 5-7).

Реалізація нескінченного циклу читання з файлу (після доходу до кінця файлу починати з початку).

Реалізація Agent на пристрої stm32 з реальними датчиками.